TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN (4 Marks)

Aim:

To recommend suitable data types, constants, and preprocessor features for a reliable electricity billing system.

Matter:

1. Recommended Data Types:

The long long data type is recommended for storing large billing totals in paise. Integer arithmetic avoids floating-point rounding errors in currency calculations.

2. Tariff Revision:

Named constants using #define can store the base charge and tariff rates. When the tariff changes, the constants can be updated and the program recompiled.

3. Limitations:

Integer data types have limited ranges and may overflow if the total exceeds their maximum values. Changing a compile-time constant requires recompilation.

4. District-Scale Suitability:

The design can handle district-scale billing when suitable data types, input validation, and overflow checks are used.

Alternative Concept and Complexity Analysis:

Tariff values can be stored in a configuration file and read when the program starts. This allows tariff revisions without recompiling the program. However, reading and validating the file adds some startup time and memory usage. Using #define is simpler, but every tariff revision requires recompilation and redistribution.

Conclusion:

Using integer paise, suitable data types, named constants, and overflow checks provides accurate and maintainable billing calculations. The design should be tested with large totals before being used at district scale.

```
#include <stdio.h>

#define BASE_CHARGE 5000LL
#define RATE1 225LL
#define RATE2 450LL

long long compute_bill(unsigned int units)
{
    long long paise = BASE_CHARGE;

    if (units > 100 && units <= 200)
        paise += (long long)(units - 100) * RATE1;
    else if (units > 200) {
        paise += 100LL * RATE1;
        paise += (long long)(units - 200) * RATE2;
    }

    return paise;
}

int main(void)
{
    unsigned int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof(tests) / sizeof(tests[0]);

    printf("Final Electricity Billing Design\n");
```

```
    for (int i = 0; i < n; i++) {
        long long bill = compute_bill(tests[i]);

        printf("%u units -> Rs %lld.%02lld\n",
               tests[i], bill / 100, bill % 100);
    }

    return 0;
}
```

